

# AFL: A Single-Round Analytic Approach for Federated Learning with Pre-trained Models

(CVPR-2025)

Run He<sup>1</sup>, Kai Tong<sup>1</sup>, Di Fang<sup>1</sup>, Han Sun<sup>2,3</sup>, Ziqian Zeng<sup>1</sup>, Haoran Li<sup>4</sup>, Tianyi Chen<sup>5</sup>, Huiping Zhuang<sup>1\*</sup> <sup>1</sup> South China University of Technology <sup>2</sup> Tsinghua University <sup>3</sup> Beijing National Research Center for Information Science and Technology <sup>4</sup> The Hong Kong University of Science and Technology <sup>5</sup> Microsoft

## Course Instructor

Prof. Dr. C. Krishna Mohan

## Presented by

Atish Kadam

CS25MTECH14003

## Assigned TA

Ishu Priya



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్  
भारतीय प्रौद्योगिकी संस्थान हैदराबाद  
Indian Institute of Technology Hyderabad

# Summary of Paper

---

## **Pain Points of Conventional FL:**

- Data Heterogeneity (non-IID)  $\rightarrow$  divergence
- Large client numbers  $\rightarrow$  performance collapse
- Slow convergence  $\rightarrow$  hundreds of rounds
- High communication cost  $\rightarrow$  bandwidth bottleneck

## **•Problem Statement**

- The research objective is to develop a novel FL framework that overcomes these limitations by providing analytical, closed-form solutions for efficient and robust training

# Summary of Paper

---

## Analytic Federated Learning

- We propose **Analytic Federated Learning (AFL)**, a gradient-free FL framework utilizing analytical closed-form solutions for both local client training and aggregation.
- AFL incorporates a **one-epoch client** training stage where a **pre-trained network's** classification head is reformulated into a localized linear regression problem, yielding an LS-based solution.
- The aggregation stage introduces an **Absolute Aggregation (AA) law**, derived analytically, enabling single-round aggregation without approximation or parameter tuning

# Summary of Paper

## 1. Absolute Aggregation Law (AAL)

$$\widehat{W} = W_u \widehat{W}_u + W_v \widehat{W}_v$$

- Global model = weighted combination of local models
- Weights depend on:
  - $C_k = X_k^T X_k$  data covariance)
- **Exact centralized solution (no raw data sharing)**

## 2. Multi-Client Extension

$$\widehat{W}_{agg,k} = W_{agg} \widehat{W}_{agg,k-1} + W_k \widehat{W}_k$$

- **Aggregation is pairwise & iterative, not averaging**

### Limitation

- Requires full column rank
  - Breaks when data is limited / rank-deficient

## 3. RI-AA Law (Fix with Regularization)

$$\widehat{W}_{agg,k} = (C_{agg,k}^r + k\gamma I)^{-1} C_{agg,k}^r \widehat{W}_{agg,k}^r$$

- **Adds regularization to handle rank deficiency**

# Summary of Paper

## ◆ Local Stage

Each client:

Compute:

$$\mathbf{X}_k, \mathbf{Y}_k$$

Solve:

$$\hat{\mathbf{W}}_k^r = (\mathbf{X}_k^T \mathbf{X}_k + \gamma \mathbf{I})^{-1} \mathbf{X}_k^T \mathbf{Y}_k$$


Compute:

$$\mathbf{C}_k^r = \mathbf{X}_k^T \mathbf{X}_k + \gamma \mathbf{I}$$



## ◆ Server Stage

Step 1: Initialize

$$\hat{\mathbf{W}}_{\text{agg},0}^r = \mathbf{0}, \mathbf{C}_{\text{agg},0}^r = \mathbf{0}$$

Step 2: Aggregate ↻

↻ For each client:

$$\hat{\mathbf{W}}_{\text{agg},k}^r = \mathbf{W}_{\text{agg}} \hat{\mathbf{W}}_{\text{agg},k-1}^r + \mathbf{W} \hat{\mathbf{K}}_k^r$$

$$\mathbf{C}_{\text{agg},K}^r = \mathbf{C}_{\text{agg},k-1}^r + \mathbf{C}_k^r$$

Step 3: Restore

$$\hat{\mathbf{W}} = (\mathbf{C}_{\text{agg},K})^{-1} \mathbf{C}_{\text{agg}} \hat{\mathbf{W}}_{\text{agg},K}^r$$

# Implementation Details

---

- **Datasets:** CIFAR-10, CIFAR-100, Tiny-ImageNet
- **Backbone:** ResNet-18 pretrained on ImageNet-1k (frozen)
- **Data Partitioning:**
  - NIID-1: Latent Dirichlet Allocation (LDA) —  $\alpha = 0.1, 0.01$
  - NIID-2: Sharding —  $s = 10, 5$  (CIFAR-100/Tiny),  $s = 4, 2$  (CIFAR-10)
- **Setup:**
  - 100 clients per experiment (also tested 500, 1000)
  - Batch size: 512

# Performance Metrics Used

---

## Primary Metric:

**Top-1 Classification Accuracy (%)** on held-out test set

## Secondary Metrics:

- **Training Time (seconds)** — wall-clock time for full pipeline
- **Communication Rounds** — number of server-client aggregation cycles

## Analysis Dimensions:

Accuracy vs.  $\alpha$  (heterogeneity level) — data heterogeneity invariance

Accuracy vs.  $K$  (client count) — client-number invariance

# Implementation - Results

Invariance to Heterogeneity with Alpha = 0.1 and Alpha = 0.05

## CIFAR - 10

Alpha = 0.1

```
Training Accuracy at training set on Client #99: 99.99999237060547%
Elapsing time for local training: 237.0807478427887%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 258.8226857185364%
Average accuracy on all Client: 82.5%
Average accuracy after regularization cleaning on all Client: 82.48999786376953%
Elapsing time plus cleansing regularization: 278.4478986263275%
written it to a csv file named cifar10resnet18_100_0.1.csv.
```

Alpha = 0.05

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 244.53295707702637%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 267.31181931495667%
Average accuracy on all Client: 82.5%
Average accuracy after regularization cleaning on all Client: 82.48999786376953%
Elapsing time plus cleansing regularization: 286.1822757720947%
written it to a csv file named cifar10resnet18_100_0.05.csv.
```

## CIFAR - 100

Alpha = 0.1

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 263.19539070129395%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 284.6139307022095%
Average accuracy on all Client: 58.5099983215332%
Average accuracy after regularization cleaning on all Client: 58.55999755859375%
Elapsing time plus cleansing regularization: 303.7576811313629%
written it to a csv file named cifar100resnet18_100_0.1.csv.
```

Alpha = 0.05

```
Training Accuracy at training set on Client #99: 99.99999237060547%
Elapsing time for local training: 250.54799056053162%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 272.4004168510437%
Average accuracy on all Client: 58.5099983215332%
Average accuracy after regularization cleaning on all Client: 58.55999755859375%
Elapsing time plus cleansing regularization: 291.2432882785797%
written it to a csv file named cifar100resnet18_100_0.05.csv.
```

## Tinyimagenet

Alpha = 0.1

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 520.7191429138184%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 544.459431886673%
Average accuracy on all Client: 54.779998779296875%
Average accuracy after regularization cleaning on all Client: 54.80999755859375%
Elapsing time plus cleansing regularization: 564.5403981208801%
written it to a csv file named tinyimagenetresnet18_100_0.1.csv.
```

Alpha = 0.05

```
Training Accuracy at training set on Client #99: 99.71098327636719%
Elapsing time for local training: 515.3877384662628%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 539.0812811851501%
Average accuracy on all Client: 54.779998779296875%
Average accuracy after regularization cleaning on all Client: 54.80999755859375%
Elapsing time plus cleansing regularization: 560.0547194480896%
written it to a csv file named tinyimagenetresnet18_100_0.05.csv.
```



# Implementation - Results

Invariance to Heterogeneity with CIFAR-10 (S = 4 and S = 2) CIFAR-100 and Tinyimagenet (S = 10 and S = 5)

CIFAR - 10

S = 4

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 238.40551161766052%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 260.2075307369232%
Average accuracy on all Client: 82.5%
Average accuracy after regularization cleaning on all Client: 82.48999786376953%
Elapsing time plus cleansing regularization: 278.283061504364%
written it to a csv file named cifar10resnet18_100_0.1.csv.
```

S = 2

```
Training Accuracy at training set on Client #99: 99.91596984863281%
Elapsing time for local training: 234.9704463481903%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 255.78718256950378%
Average accuracy on all Client: 82.5%
Average accuracy after regularization cleaning on all Client: 82.48999786376953%
Elapsing time plus cleansing regularization: 274.6025056838989%
written it to a csv file named cifar10resnet18_100_0.1.csv.
```

CIFAR - 100

S = 10

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 277.7598855495453%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 299.43948769569397%
Average accuracy on all Client: 58.5099983215332%
Average accuracy after regularization cleaning on all Client: 58.55999755859375%
Elapsing time plus cleansing regularization: 317.6767156124115%
written it to a csv file named cifar100resnet18_100_0.1.csv.
```

S = 5

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 269.23117446899414%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 292.1030867099762%
Average accuracy on all Client: 58.5099983215332%
Average accuracy after regularization cleaning on all Client: 58.55999755859375%
Elapsing time plus cleansing regularization: 312.10649275779724%
written it to a csv file named cifar100resnet18_100_0.1.csv.
```

Tinyimagenet

S = 10

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 510.8326425552368%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 535.0601077079773%
Average accuracy on all Client: 54.779998779296875%
Average accuracy after regularization cleaning on all Client: 54.80999755859375%
Elapsing time plus cleansing regularization: 556.571320772171%
written it to a csv file named tinyimagenetresnet18_100_0.1.csv.
```

S = 5

```
Training Accuracy at training set on Client #99: 100.0%
Elapsing time for local training: 652.6979072093964%
Aggregating!
Aggregation done!
Evaluating global model!
Elapsing time for training and aggregation: 681.893501996994%
Average accuracy on all Client: 54.779998779296875%
Average accuracy after regularization cleaning on all Client: 54.80999755859375%
Elapsing time plus cleansing regularization: 703.9910371303558%
written it to a csv file named tinyimagenetresnet18_100_0.1.csv.
```

# Implementation - Results

Invariance to No. of Clients with  $K = 100$ ,  $K = 500$  and  $K = 1000$

## CIFAR - 100

$K = 500$

```
Elapsing time for local training: 797.8273799419403%  
Aggregating!  
Aggregation done!  
Evaluating global model!  
Elapsing time for training and aggregation: 841.1920909881592%  
Average accuracy on all Client: 58.32999801635742%  
Average accuracy after regularization cleaning on all Client: 58.55999755859375%  
Elapsing time plus cleansing regularization: 859.4695720672607%  
written it to a csv file named cifar100resnet18_500_0.1.csv.
```

$K = 1000$

```
Training Accuracy at training set on Client #999: 100.0%  
Elapsing time for local training: 826.0462906360626%  
Aggregating!  
Aggregation done!  
Evaluating global model!  
Elapsing time for training and aggregation: 879.507586479187%  
Average accuracy on all Client: 58.14999771118164%  
Average accuracy after regularization cleaning on all Client: 58.55999755859375%  
Elapsing time plus cleansing regularization: 898.1180558204651%  
written it to a csv file named cifar100resnet18_1000_0.1.csv.
```

## Tinyimagenet

$K = 500$

```
Elapsing time for local training: 1163.0233738422394%  
Aggregating!  
Aggregation done!  
Evaluating global model!  
Elapsing time for training and aggregation: 1206.4319450855255%  
Average accuracy on all Client: 54.75%  
Average accuracy after regularization cleaning on all Client: 54.80999755859375%  
Elapsing time plus cleansing regularization: 1231.3856890201569%  
written it to a csv file named tinyimagenetresnet18_500_0.1.csv.
```

$K = 1000$

```
Training Accuracy at training set on Client #999: 100.0%  
Elapsing time for local training: 1120.0678660869598%  
Aggregating!  
Aggregation done!  
Evaluating global model!  
Elapsing time for training and aggregation: 1176.8097696304321%  
Average accuracy on all Client: 54.71999740600586%  
Average accuracy after regularization cleaning on all Client: 54.80999755859375%  
Elapsing time plus cleansing regularization: 1198.2075035572052%  
written it to a csv file named tinyimagenetresnet18_1000_0.1.csv.
```

# Reproduced Result

## Invariance to Heterogeneity

| Dateset       | Setting      | Accuracy |
|---------------|--------------|----------|
| CIFAR - 10    | Alpha = 0.1  | 82.5%    |
|               | Alpha = 0.05 | 82.5%    |
|               | S = 4        | 82.5%    |
|               | S = 2        | 82.5%    |
| CIFAR - 100   | Alpha = 0.1  | 58.50%   |
|               | Alpha = 0.05 | 58.50%   |
|               | S = 10       | 58.50%   |
|               | S = 5        | 58.50%   |
| Tiny-ImageNet | Alpha = 0.1  | 54.80%   |
|               | Alpha = 0.05 | 54.80%   |
|               | S = 10       | 54.80%   |
|               | S = 5        | 54.80%   |

## Invariance to Number of Clients

| Dateset       | No. of Clients | Accuracy |
|---------------|----------------|----------|
| CIFAR - 100   | K = 100        | 58.50%   |
|               | K = 500        | 58.50%   |
|               | K = 1000       | 58.50%   |
| Tiny-ImageNet | K = 100        | 54.80%   |
|               | K = 500        | 54.80%   |
|               | K = 1000       | 54.80%   |

# Novelty Implemented

## Gap in the Original AFL Paper

**Key Insight Motivating Compression:** In non-IID federated learning, each client sees only a small subset of classes. The local weight matrix  $w$  therefore has low intrinsic rank — most singular values are near zero. Transmitting the full matrix wastes bandwidth on near-zero components.

| Problem                          | Detail                                                                                                                            |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Full-rank $W$ always transmitted | Even when clients hold very few classes (extreme non-IID), the full $\text{feat\_size} \times \text{num\_classes}$ matrix is sent |
| $W$ grows with model scale       | For ViT-B16 ( $\text{feat\_size} = 768$ ) with 200 classes, $W$ per client = $0.234 \text{ MB} \times K$ clients                  |
| No compression mechanism         | The original code has no way to trade off accuracy vs. communication cost                                                         |
| Silent rank-deficiency crashes   | When $N_k < \text{feat\_size}$ (few samples per client), direct inversion fails with no fallback                                  |

# Novelty Implemented

## Rank-r SVD Theory

- $W = U \cdot \Sigma \cdot V^T$
- where  $U \in \mathbb{R}^{m \times m}$  is orthogonal (left singular vectors),  $\Sigma \in \mathbb{R}^{m \times n}$  is diagonal with non-negative singular values  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ , and  $V^T \in \mathbb{R}^{n \times n}$  is orthogonal (right singular vectors).
- The singular values tell you how much "energy" each component carries. If the matrix is truly rank-3, only  $\sigma_1, \sigma_2, \sigma_3$  are non-zero. If it's approximately rank-3,  $\sigma_1, \sigma_2, \sigma_3$  are large and  $\sigma_4, \sigma_5, \dots, \sigma_{100} \approx 0$ .
- Singular Values =  $\sqrt{\text{eigenvalues of } W^T W}$
- Big eigenvalue  $\rightarrow$  big singular value  $\rightarrow$  important direction  $\rightarrow$  keep it.

## Truncated (Rank-r) SVD keeps only the top-r components:

- $W \approx U_r \cdot \text{diag}(S_r) \cdot V_{h_r}$
- where  $U_r \in \mathbb{R}^{512 \times r}$ ,  $S_r \in \mathbb{R}^r$ ,  $V_{h_r} \in \mathbb{R}^{r \times 100}$ . This is provably the best rank-r approximation of  $W$  in the Frobenius norm sense the Eckart–Young theorem guarantees this. We are not making an arbitrary approximation; we are making the optimal one.
- We consider only Top  $r$  columns of  $U$ , Top  $r$  values of  $\Sigma$  and Top  $r$  rows of  $V^T$ .

# Novelty Implemented

## Rank-r SVD Compression + Communication Cost Analyzer in AFL

### Rank-r SVD Compression

- After each client computes its local weight matrix  $w$  (shape:  $feat\_size \times num\_classes$ ), truncated SVD is applied before aggregation.
- Only the top- $r$  singular components are retained:  $W \approx U_r \cdot diag(S_r) \cdot Vh_r$
- The matrix is reconstructed on the server side — the aggregation logic itself is unchanged
- Controlled by a single new argument: `--rank_r`

### Communication Cost Analyzer

- Reports original vs. compressed cost, MB saved, and percentage savings

# Novelty Implemented

## CIFAR - 100

K = 100

```
Client 99 Train Acc: 94.43%
Local training time: 345.89s

Aggregating!
Aggregation done!

=====
COMMUNICATION COST REPORT
=====
Embedding size (feat) : 512
Num classes           : 100
Num clients (K)       : 100
Rank-r compression    : 32 (effective = 32)
C matrix dominance    : 83.7% of total per-client cost

=====
                        Original Compressed
=====
W matrix (per client)  0.3906MB  0.1497MB
C matrix (per client)  2.0000MB  2.0000MB
Total (per client)     2.3906MB  2.1497MB

=====
TOTAL (all clients)    239.062MB  214.966MB
W-only savings         61.7%
Overall savings        10.1% (24.097 MB saved)

=====

Evaluating global model!
Total time (train + agg): 367.63s
Global Accuracy: 58.53%

Cleaning regularization...
Accuracy after cleaning: 58.60%
Total time (with cleaning): 386.36s
Saved results to cifar100_resnet18_100_0.1.csv
```

K = 500

```
Client 499 Train Acc: 94.31%
Local training time: 536.41s

Aggregating!
Aggregation done!

=====
COMMUNICATION COST REPORT
=====
Embedding size (feat) : 512
Num classes           : 100
Num clients (K)       : 500
Rank-r compression    : 32 (effective = 32)
C matrix dominance    : 83.7% of total per-client cost

=====
                        Original Compressed
=====
W matrix (per client)  0.3906MB  0.1497MB
C matrix (per client)  2.0000MB  2.0000MB
Total (per client)     2.3906MB  2.1497MB

=====
TOTAL (all clients)    1195.312MB  1074.829MB
W-only savings         61.7%
Overall savings        10.1% (120.483 MB saved)

=====

Evaluating global model!
Total time (train + agg): 571.25s
Global Accuracy: 58.37%

Cleaning regularization...
Accuracy after cleaning: 58.50%
Total time (with cleaning): 591.70s
Saved results to cifar100_resnet18_500_0.1.csv
```

K = 1000

```
Client 999 Train Acc: 100.00%
Local training time: 754.36s

Aggregating!
Aggregation done!

=====
COMMUNICATION COST REPORT
=====
Embedding size (feat) : 512
Num classes           : 100
Num clients (K)       : 1000
Rank-r compression    : 32 (effective = 32)
C matrix dominance    : 83.7% of total per-client cost

=====
                        Original Compressed
=====
W matrix (per client)  0.3906MB  0.1497MB
C matrix (per client)  2.0000MB  2.0000MB
Total (per client)     2.3906MB  2.1497MB

=====
TOTAL (all clients)    2390.625MB  2149.658MB
W-only savings         61.7%
Overall savings        10.1% (240.967 MB saved)

=====

Evaluating global model!
Total time (train + agg): 798.11s
Global Accuracy: 58.21%

Cleaning regularization...
Accuracy after cleaning: 58.62%
Total time (with cleaning): 814.27s
Saved results to cifar100_resnet18_1000_0.1.csv
```



# Novelty Implemented

## Tiny-Imagenet

K = 100

Client 99 Train Acc: 81.98%  
Local training time: 627.27s

Aggregating!  
Aggregation done!

### COMMUNICATION COST REPORT

Embedding size (feat) : 512  
Num classes : 200  
Num clients (K) : 100  
Rank-r compression : 32 (effective = 32)  
C matrix dominance : 71.9% of total per-client cost

|                       | Original  | Compressed        |
|-----------------------|-----------|-------------------|
| W matrix (per client) | 0.7812MB  | 0.1741MB          |
| C matrix (per client) | 2.0000MB  | 2.0000MB          |
| Total (per client)    | 2.7812MB  | 2.1741MB          |
| -----                 |           |                   |
| TOTAL (all clients)   | 278.125MB | 217.407MB         |
| W-only savings        | 77.7%     |                   |
| Overall savings       | 21.8%     | (60.718 MB saved) |

Evaluating global model!  
Total time (train + agg): 650.83s  
Global Accuracy: 54.97%

Cleaning regularization...  
Accuracy after cleaning: 54.97%  
Total time (with cleaning): 668.88s  
Saved results to tinyimagenet\_resnet18\_100\_0.1.csv

K = 500

Client 499 Train Acc: 84.19%  
Local training time: 928.11s

Aggregating!  
Aggregation done!

### COMMUNICATION COST REPORT

Embedding size (feat) : 512  
Num classes : 200  
Num clients (K) : 500  
Rank-r compression : 32 (effective = 32)  
C matrix dominance : 71.9% of total per-client cost

|                       | Original   | Compressed         |
|-----------------------|------------|--------------------|
| W matrix (per client) | 0.7812MB   | 0.1741MB           |
| C matrix (per client) | 2.0000MB   | 2.0000MB           |
| Total (per client)    | 2.7812MB   | 2.1741MB           |
| -----                 |            |                    |
| TOTAL (all clients)   | 1390.625MB | 1087.036MB         |
| W-only savings        | 77.7%      |                    |
| Overall savings       | 21.8%      | (303.589 MB saved) |

Evaluating global model!  
Total time (train + agg): 966.48s  
Global Accuracy: 54.84%

Cleaning regularization...  
Accuracy after cleaning: 54.94%  
Total time (with cleaning): 988.81s  
Saved results to tinyimagenet\_resnet18\_500\_0.1.csv

K = 1000

Client 999 Train Acc: 94.84%  
Local training time: 1218.96s

Aggregating!  
Aggregation done!

### COMMUNICATION COST REPORT

Embedding size (feat) : 512  
Num classes : 200  
Num clients (K) : 1000  
Rank-r compression : 32 (effective = 32)  
C matrix dominance : 71.9% of total per-client cost

|                       | Original   | Compressed         |
|-----------------------|------------|--------------------|
| W matrix (per client) | 0.7812MB   | 0.1741MB           |
| C matrix (per client) | 2.0000MB   | 2.0000MB           |
| Total (per client)    | 2.7812MB   | 2.1741MB           |
| -----                 |            |                    |
| TOTAL (all clients)   | 2781.250MB | 2174.072MB         |
| W-only savings        | 77.7%      |                    |
| Overall savings       | 21.8%      | (607.178 MB saved) |

Evaluating global model!  
Total time (train + agg): 1269.52s  
Global Accuracy: 54.94%

Cleaning regularization...  
Accuracy after cleaning: 55.00%  
Total time (with cleaning): 1288.84s  
Saved results to tinyimagenet\_resnet18\_1000\_0.1.csv



# Novelty Implemented

| Dateset       | Setting  | Accuracy | Communication cost saving on W | Percentage Saving on W |
|---------------|----------|----------|--------------------------------|------------------------|
| CIFAR - 100   | K = 100  | 58.50%   | 24.097 MB                      | 61.7%                  |
|               | K = 500  | 58.50%   | 120.483 MB                     | 61.7%                  |
|               | K = 1000 | 58.50%   | 2149.658 MB                    | 61.7%                  |
| Tiny Imagenet | K = 100  | 54.80%   | 60.718 MB                      | 77.7%                  |
|               | K = 500  | 54.80%   | 303.589 MB                     | 77.7%                  |
|               | K = 1000 | 54.80%   | 607.178 MB                     | 77.7%                  |

# Conclusion and Future Work

## My contribution extends AFL with:

Rank-r SVD compression on local weight matrices — reducing per-client upload cost by up to 87% + Communication cost analysis.

## Three open directions identified from our analysis:

- 1. Compress the C matrix** — The Gram matrix ( $feat\_size \times feat\_size$ ) dominates transmission cost (4.00 MB vs 0.39 MB for W in ResNet-18/CIFAR-100). C gets filled everywhere. No empty columns. No obvious sparsity. No clean low-rank structure.
- 2. Partial participation and stragglers** — Adapting the AA law to handle asynchronous or dropout clients without sacrificing the invariance property is an open problem.
- 3. Non-linear extension** — AFL is built on linear classifiers. Incorporating non-linear projections, kernel functions, or layer-wise least-squares formulations (label projection) could extend AFL to settings where the embedding space is not linearly separable.

# References

---

1. Khodak et al. — *Federated Learning with Only Positive Labels* — uses low-rank structure in client updates. ICML 2022.
2. Zhao et al. — *FedPAGE: A Fast Local Stochastic Gradient Method for Communication-Efficient Federated Learning* — analyzes low-rank gradient compression. NeurIPS 2021.
3. Vogels et al. — *PowerSGD: Practical Low-Rank Gradient Compression for Distributed Optimization* — proposes rank- $r$  approximation of gradient matrices via SVD for distributed training. NeurIPS 2019.

**Thank You**